

LLM-QGraph: Threat Intelligence Query Graph Construction with Large Language Models

Hongfa Yang^{†‡§}, Yu Wen^{*†§}, Dan Meng^{†§}

[†]Institute of Information Engineering, Chinese Academy of Sciences, P.R. China

[‡]School of Cyber Security, University of Chinese Academy of Sciences, P.R. China

[§]Key Laboratory of Cyberspace Security Defense, P.R. China

{yanghongfa, wenyu, mengdan}@iie.ac.cn

*Corresponding author

Abstract—Advanced Persistent Threats (APTs) and other organized and targeted cyber threats pose significant risks to critical areas such as national politics and economy. Security personnel need to analyze threat intelligence to hunt suspicious behaviors in the system. Existing threat intelligence graph construction methods have limitations such as insufficient temporal continuity and low accuracy in constructing threat intelligence knowledge graphs. To address these issues, this paper proposes an automated method named LLM-QGraph for constructing threat intelligence query graphs based on LLMs. This method designs appropriate prompts to fine-tune the Llama3 model, enabling it to generate log sequences that describe the attack steps according to CTI, thereby compensating for the neglect of temporal relationships. Threat intelligence query graphs are constructed through an improved provenance graph algorithm to enhance the performance of threat hunting. Experiments are conducted on four real-world datasets. The results show that compared with other graph-based methods, the query graphs constructed by this method are more effective in threat hunting applications.

Index Terms—cyber threat hunting, large language models, cyber threat intelligence

I. INTRODUCTION

Advanced Persistent Threat (APT) refers to organized and targeted cyber threats that have emerged in recent years. These stealthy attacks pose significant risks to critical areas such as national politics and economics [1]. Security experts often learn about attacks occurring within peer companies through blogs, forums, and threat intelligence (TI) reports, which outline the steps attackers might take and provide some IOCs. However, experts remain uncertain whether similar attacks have occurred in their own systems. Suppose that known attacks already exist in the system. Utilizing the characteristics of these attack behaviors provides a solution for hunting possible threat behaviors. Analyzing threat intelligence and extracting accurate attack information is of importance for security personnel to hunt suspicious behaviors in the system.

Threat Hunting is divided into two steps: creating specific queries based on the obtained attack information and efficiently capturing threats in the system using the created queries. Creating queries corresponds to specific behaviors to be investigated. In contrast to the methods used in studies [2], [3], which relied on manually generated query graphs from incident reports, Satvat et al. [4] employ a natural language processing pipeline to identify relationships between system entities in Cyber threat intelligence (CTI), ultimately extract-

ing query graphs using this information. With the emergence of large language models (LLMs), AttackKG+ [5] and LLM-TIKG [6] methods utilize LLMs to extract Indicators of Compromise (IOCs) from CTI and construct knowledge graphs by combining with The Tactics, Techniques, and Procedures (TTPs) framework proposed by MITRE ATT&CK [7].

The current research about threat Hunting, after using provenance tools to construct causal graphs from system logs, subgraph matching is carried out using query graphs to hunt suspicious attack behaviors in the system. Despite their advance in constructing the threat intelligence knowledge graph, these approaches for threat hunting suffer from the following limitations:

- **Temporal Continuity:** TTPs mainly focus on the logical sequence of tactics and techniques and do not provide precise time measurement information. These TTPs-based knowledge graphs have isolated points. In addition, Provenance tools attach great importance to the time relationship when constructing graphs based on logs. The threat intelligence knowledge graph cannot maintain temporal continuity. Thus, the query graphs extracted from CTI should maintain it.
- **Accuracy of threat hunting:** Due to the significant differences in the methods of constructing knowledge graphs by [4]–[6], as well as in the methods of constructing provenance graphs and the graph properties of both, the accuracy of threat hunting using these knowledge graphs is relatively low, and current methods for threat hunting still relies on experts to manually construct query graphs.

Inspired by how provenance tools construct graphs using log sequences [8], [9], given that CTI essentially describes attack behaviors, is it possible to generate system log sequences by simulating the attack behaviors described in CTI, and then construct query graphs? The breakthroughs of LLMs [10], [11] shed light on these issues. Due to their powerful language comprehension ability, LLMs can thoroughly analyze various elements in CTI, including attack methods, attacker characteristics, affected systems, and other information. Secondly, through learning from massive amounts of text data, large language models can gain an in-depth understanding of the semantic, syntactic, and pragmatic rules of language. This enables them to generate content that is logically coherent,

semantically accurate, and compliant with rules when given a specific topic or prompt.

We propose the LLM-QGraph method, which utilizes LLMs to generate log sequences of the attack behaviors based on CTI, and then constructs query graphs from the log sequences through the improved provenance graph algorithm. Firstly, we use regular expression matching to extract text, IOCs, and time from CTI. Then, by designing appropriate prompts and leveraging the few-shot learning ability of GPT, we utilize the text, IOCs, and time to generate a fine-tuned log dataset. we use the LoRA-based instruction tuning method to fine-tune the large model, the Llama3-8B-Instruction [12] model, enabling the large model to generate log sequences in the specified format that describe attack behaviors according to CTI. The log sequences contain the temporal relationships among the IOCs, thus maintaining the temporal continuity of the query graphs constructed using them. Finally, we improve the provenance graph algorithm. We extract different system entities from the log sequences and associate all these system entities according to the time sequence of the log sequences to construct query graphs. In this way, the query graphs and the provenance graphs construction algorithm maintain consistency in terms of construction and properties, reducing the need for manual intervention. We crawl 200 pieces of threat intelligence to fine-tune the LLM, and then use the constructed query graphs on real-world log datasets to hunt threats and verify their performance. The contributions of this paper are as follows:

- To the best of our knowledge, we are the first to utilize large models for log sequence generation and preserve the temporal relationships among IOCs.
- We improve the provenance graph method and maintain consistency in the construction methods of query graphs and provenance graphs, thereby improving the accuracy of threat hunting.
- We conduct experiments on four real-world datasets. The performance of threat hunting using the query graphs constructed by LLM-Qgraph outperforms those constructed by other graph-based methods.

In the rest of this paper, Section II introduces the related work, Section III presents the detailed methodology of LLM-QGraph, Section IV evaluates the experiment, and Section V summarizes the paper.

II. RELATED WORK

A. Threat Hunting

Hypothesis-driven threat hunting technology is a method that actively discovers potential threats by leveraging known attack information. The sources from which threat hunters obtain attack information include security blogs, forums, and TI, among others. Mirajerdi et al. [2] propose a non-learning-based method named “Poirot”, which manually creates query graphs for known advanced persistent threat (APT) attack behaviors and uses them to facilitate the capture of the provenance graph. With the development of graph neural

networks, in order to efficiently hunt subgraphs related query graphs in large-scale provenance graphs, Altinisik et al. [13] proposed ProvG-Searcher, which divides the provenance graph into subgraphs by extracting ego-graph structures and then introduces a graph neural network model to model the sequential relationships between query graphs and provenance subgraphs. However, the accuracy and running efficiency of these methods are limited by the creation of query graphs. As the scale of the provenance graph increases, its performance will decline. In these research, the way of creating query graphs is also manually created by experts. Therefore, this paper attempts to automate the creation of query graphs from CTI and effectively extract attack features, thus improving the accuracy of threat hunting.

B. CTI Extraction

CTI plays a crucial role in the security offense and defense battle to keep up with the rapidly evolving cyber-attack landscape [14], [15]. Current CTI extraction work in terms of information granularity includes three aspects: CTI report extraction, unstructured information extraction, and TTP identification. iACE [16] propose a graph mining technique to collect IoCs from tens of thousands of security articles. Extractor [4] and ThreatRaptor [17] customize Natural Language Processing (NLP) techniques to model attack behaviors in text as attack graphs. TTPDrill [18] and rcATT [19] derive threat behaviors from reports and map them to attack patterns (such as the tactics and techniques in MITRE ATT&CK) using predefined ontologies or machine learning models. Similar to previous studies, the main work of AttackKG [20] is to achieve automatic knowledge extraction from CTI. However, what distinguishes AttackKG from these works is that it identifies TTPs and constructs a Technical Knowledge Graph (TKG) to aggregate technical-level knowledge across CTI. These CTI extraction works focus on IOCs entity extraction and TTPs identification in terms of information granularity and cannot maintain temporal continuity, resulting in discrete nodes when constructing graphs. In this paper, more consideration is given to the temporal relationships between entities when constructing graphs to maintain the connectivity of the query graph.

C. LLMs for Cyber Security

Currently, scholars have been extensively exploring the use of LLMs in the field of cybersecurity. Maxime et al. [21] used LLMs to extract knowledge entities from cybersecurity-related texts. AttackKG+ [5] and LLM-TIKG [6] methods analyzed threat intelligence based on LLMs and combined TTPs to construct attack knowledge graphs. LLMs are pre-trained on massive datasets containing diverse content such as code and log data, which makes them particularly adept at specific tasks like log parsing [22]. The significant advantage of large models in text comprehension makes it possible to generate log sequences with CTI.

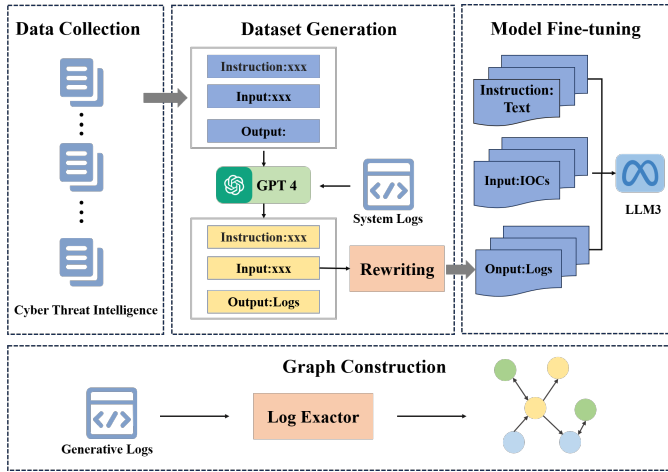


Fig. 1: System Overview

III. METHODOLOGY

As shown in Figure. 1, our system consists of four main components: data collection, data generation, model fine-tuning and graph construction. Firstly, we collect data on cyber threat intelligence websites through the method of regular expression matching. Secondly, by leveraging the powerful few-shot learning ability of Chat-GPT, we designed task instructions and output examples to obtain the datasets required by this method. Then, we used the LoRA-based instruction tuning method (Hu et al., 2021) to fine-tune the Llama3-8B-Instruction model, enabling it to analyze the input text, combine the Indicators of Compromise (IOCs) and time information, and output this information in the format of system logs. Finally, we improved the provenance graph algorithm to construct time-related graphs based on the generated system logs.

A. Data Collection

To collect data, we design web crawlers targeting open-source threat platforms (including DFIR Lab Threat Reports and MISP Open Source Threat Intelligence and Sharing Platform). We scrape the text content and pictures of each report and remove irrelevant information such as advertisements and sidebars. These reports use subtitles to divide different attack stages, and each section describes the specific steps of the attack from different attack stages. The DFIR Lab Threat Reports contain text descriptions of each attack stage as well as the timeline of the entire attack process. We split them according to the text of each attack stage, extract the timeline and system instructions of each stage, and store the datasets in the form of "text + time + instruction" to facilitate the use of large models to generate more fine-tuning datasets. There are a total of 12 attack stages among them: Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, Impact.

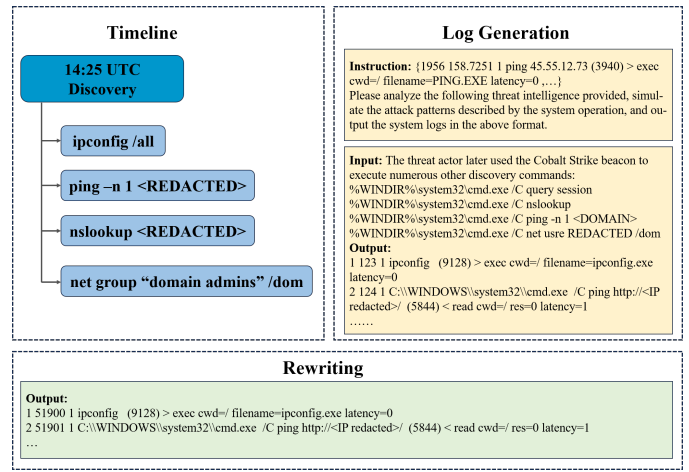


Fig. 2: The prompt for data generation with instruction, examples and input.

B. Dataset Generation

Regarding the method for obtaining the output in the dataset, that is, logs, the most straightforward approach is to simulate the attacks described in threat intelligence within the system and continuously collect system logs to serve as the output for model fine-tuning. However, this method requires experts to have an understanding of various attacks and a significant amount of time to reproduce them. Considering the powerful few-shot learning ability of GPT, it can assist us in generating datasets for model fine-tuning. This entails setting instructions and examples for different tasks, combining them with the input text, and then providing them to the model. The model generate logs in the target format, and these logs serve as the "output" in the fine-tuning dataset. As shown in Figure 2, we select the "GPT-4.0-turbo" model and use the "instruction + example + input" template. The instruction describes the generation task, the example is a sequence of system logs containing real attacks, which is used to explain the task in the instruction and represent the expected output structure, while the input represents the threat intelligence to be analyzed. Since the original large model has limited understanding of logs, we manually rewrite the log statements output by the model, and supplement the log sequences, time information, process information, file information, etc. with the reports. For example, the timestamp is rewritten by converting it into milliseconds according to the timeline in the CTI. Finally, the "threat intelligence + Indicators of Compromise (IOCs) + rewritten log sequences" are combined to form the dataset for fine-tuning the LLM.

C. Model Fine-tuning

To create a large language model (LLM) customized for specific tasks, we conduct instruction tuning on Meta's Llama3-8B-Instruction model [Llama3] using the datasets collected above. The Llama3-8B-Instruction is an 8B-parameter model in the Llama 3 model series that is specifically designed to follow instructions. By performing instruction tuning on the

basis of Llama3-8B, it can better understand and execute human instructions and demonstrates good performance in a variety of natural language processing tasks, providing users with high-quality language interaction services. Instruction tuning is a technique used to fine-tune pre-trained large language models, which employs a carefully selected collection of natural language examples. This method enables these large language models to utilize their effective generalization ability for unseen tasks and optimize their performance for specific objectives. Since large language models consist of a large number of model parameters, optimizing all parameters is costly. This issue can be resolved by training only a small portion of the parameters, which can be either a subset of the existing model parameters or a newly added set of parameters. This approach alleviates the resource intensiveness of traditional fine-tuning techniques.

Here, Low-Rank Adaptation (LoRA) (Hu et al., 2021) is adopted to fine-tune the model. Considering the case of optimizing the parameter matrix W , the update process can be generally written as: $W \leftarrow W_0 + \Delta W$. The basic idea of LoRA is to freeze the original matrix $W_0 \in R^{m*n}$ and only update the ΔW parameter. This update process can be represented as shown in Eq. (1), where only A and B are the training parameters. In the forward process, both W_0 and ΔW are multiplied by the same input and then added together, as shown in Eq. (2):

$$W_0 + \Delta W = W_0 + BA, B \in R^{m*r}, A \in R^{r*n} \quad (1)$$

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (2)$$

The Llama3-8B-Instruction model is fine-tuned using the collected datasets so that it could output results corresponding to the target instructions. The data format of the training dataset is "instruction + input + output", where the instruction was a natural language explanation of the task, the input is the corresponding segment to be processed, and the output is the expected response. In the log generation task, the instruction also contain several output examples as well as the format of the log output to obtain logs in the specified format. Then, the trained model is used to analyze the collected threat intelligence reports.

D. Query Graph Construction

By utilizing the fine-tuned model, we can generate a log set from the collected textual threat intelligence combined with Indicators of Compromise (IOCs). As shown in Figure 3, the format of the logs includes serial numbers, timestamps, process information, operation types, file information or IP addresses. Among them, the operation types cover reading, writing, execution, receiving, sending, and so on. Firstly, we will extract each field from the log sequence using regular expressions.

```
1 2024-1-1-20:23:10 1 C:\Program_Files\Mozilla_Firefox\firefox.exe (9892) < recvfrom cwd=/
res=1460 latency=1
2 2024-1-1-20:23:11 1 C:\Program_Files\Mozilla_Firefox\firefox.exe (9892) < recvfrom cwd=/
fd=4(<4u>johnshimkus.com->216.73.86.152:80) latency=0
...
43 2024-1-3-13:28:28 1 C:\Program_Files\7z.exe (9892) > read cwd=/
fd=4(<->C:\Users\admin\AppData\Local\Mozilla\Firefox\Profiles\John Shimkus.zip) latency=0
...
```

Fig. 3: The format of the logs.

Algorithm 1 Query Graph Construction

Require: Log file

Ensure: Directed graph

```
1: Initialize an empty graph graph and a dictionary
   label_to_index;
2: Create node_set from the log file;
3: for each Node in node_set do
4:   Add process node to graph if not exists, update
   label_to_index;
5:   if Node has a network label then
6:     Add network node and edge to graph if not
     exists;
7:   end if
8:   if Node has a filename label then
9:     Add filename node and edge to graph if not
     exists;
10:  end if
11: end for
12: Sort node_set by timestamp;
13: for each Consecutive nodes in sorted node_set do
14:   Add edge between their process nodes in graph;
15: end for
16: return graph;
```

Then, as shown in Algorithm 1, we propose an algorithm to construct a query graph using the fields extracted from the logs. Firstly, construct the *node_set* and *label_to_index* (the set of node attributes) based on all the node information extracted from the log file. Then, traverse the node set. If it is a process node and the process node in the current log has not been added to the graph yet, add it as a process node and update *label_to_index*. If the log contains network resources (such as IP addresses) and the corresponding network node has not been added yet, add it to the graph and create an edge from the process node to the network node. Similarly, if the log contains file information and the corresponding file node has not been added yet, add it to the graph and create an edge from the process node to the file node. Then, construct edges for all the process nodes in chronological order. During the process of constructing edges, the operation type is recorded for each edge. Since there are duplicate nodes in the graph, we merge the duplicate nodes within a short period of time and retain the edges with the earliest and latest times. Finally, a query graph that can represent event correlations is formed.

IV. EVALUATION

In this section, we focus on evaluating the performance and construction efficiency of the query graphs constructed by LLM-QGraph based on the parsing of CTI in threat hunting. Meanwhile, we assess the quality of the constructed query graphs through manual analysis. Specifically, our evaluations aim to answer the following research questions:

- The performance of the constructed query graphs in threat capture.
- The performance on real datasets.
- The impact of fine-tuning settings on the construction of query graphs.

A. Experimental setup

We collect 200 pieces of CTI from the DFIR Lab. After splitting the reports by attack stages, there are a total of 2,400 pieces of content in the dataset. A total of 7,200 log sequences are obtained using the above-mentioned method. The GPT selected for log generation is GPT-4.0-turbo. The model fine-tuning process is implemented using Python 3.9 on a 16GB Tesla T4 GPU. The maximum learning rate is set to 2×10^{-4} , the maximum number of steps is 120, and the maximum sequence length is set to 2,048 tokens.

TABLE I: The results of LLM-QGraph for threat hunting

Method	Dataset	Metrics			
		Acc.	Prec.	Recall	F1
Manual	Cadets	99.78	99.96	99.61	99.76
	Trace	99.34	99.98	98.69	99.33
	Theia	99.83	99.98	99.69	99.84
	FiveDirections	95.86	94.89	95.98	95.80
LLM-QGraph	Cadets	79.98	81.82	80.91	81.36
	Trace	79.99	80.94	80.43	80.68
	Theia	80.11	81.96	81.05	81.50
	FiveDirections	85.93	84.46	85.16	84.80

B. Performance of the Constructed Query Graphs in Threat Hunting

To evaluate the performance of the query graphs constructed by LLM-QGraph in threat hunting, we utilize the threat intelligence provided by the Darpa dataset to construct query graphs. Cadets, Trace, and Theia are Linux system logs, while FiveDirections is a Windows system log. Then, we reproduce the ProvG method to perform subgraph matching in the system logs and hunt the threat behaviors related to the query graphs. The accuracy, precision, recall, and F1 score are adopted as the evaluation metrics for this experiment. The experimental results are shown in Table I. LLM-QGraph performs excellently on the Cadets, Trace, and Theia datasets, with an average accuracy of 80.02%, an average precision of 81.57%, an average recall of 80.79%, and an average F1

score of 80.22%. Since the threat intelligence in the dataset used for fine-tuning LLM-QGraph comes from the Windows system, its performance on FiveDirections is higher than that on the other three datasets in terms of each indicator (by approximately 6%, approximately 3%, approximately 4%, and approximately 3%). Therefore, we believe that with sufficient data and sufficient training for fine-tuning the large model, LLM-QGraph will perform better on various datasets. Because the attacks described in the threat intelligence of the DFIR Lab are not exactly the same type of attacks as those in the Darpa dataset, and there are also differences in the textual descriptions. To some extent, this experiment also proves the effectiveness of LLM-QGraph in constructing query graphs for threat intelligence that describes new types of attacks.

TABLE II: Performance Comparison Across Methods

Method	Rate (%)	Similarity (%)
Exactor	62.18	57.15
AttacKG	70.64	60.33
AttacKG+	82.66	79.98
LLM-TINK	85.94	80.14
LLM-QGraph	80.14	86.93

C. Performance on Real Datasets

To further evaluate the performance of the query graphs constructed by LLM-QGraph, we compare LLM-QGraph with other works on threat intelligence analysis. Among them, Exactor and AttacKG analyze cyber threat reports based on NLP, while AttacKG+ and LLM-TIKG are based on LLMs. After reproducing these methods, we conduct tests on the FiveDirection dataset in Darpa. After analyzing the CTI to construct query graphs, we adopt the ProvG method to match the subgraphs of the attack behaviors related to the query graphs in the logs. According to the ratio of the number of nodes in the query graph to the number of real attack nodes in the matched provenance subgraph, which is recorded as Rate, and the vector similarity between the query graph and the matched provenance subgraph, which is recorded as Similarity, these are used as the evaluation metrics. The experimental results are shown in Table II. Our method outperforms the query graphs constructed by Exactor (by approximately 30%), AttacKG (by approximately 26%), AttacKG+ (by approximately 7%), and LLM-TINK (by approximately 7%) in terms of vector similarity. However, in terms of the number of matched nodes, its performance is lower than that of AttacKG+ (by approximately 3%) and LLM-TINK (by approximately 6%). The reason is that the existing works focus on the information granularity of analyzing threat intelligence on the entities and Indicators of Compromise (IOCs) information existing in the intelligence, ignoring the temporal correlations of these information. Therefore, there are situations where the structure of the constructed query graph is inconsistent with that of the provenance subgraph.

D. Fine-tuning Settings

During the fine-tuning process of the large model, the *max_step_length* of the model parameters and the rank of LoRA directly affect the performance of the large model and the training time. The longer the number of training steps and the higher the rank are, the better the performance of the model will be, but the longer the training time will be. For the fine-tuning process of a relatively large-scale threat intelligence dataset, we need to know which number of training epochs and rank are the most appropriate values. As shown in Figure 4 (a), when the *max_step_length* of the model is below 120, the performance of the model is gradually improving. After it exceeds 120, the performance of the model tends to stabilize, but the training time of the model is gradually increasing. Therefore, we selected 120 as the optimal *max_step_length* for the model fine-tuning. Figure 4 (b) shows that the performance of the large model does not fluctuate significantly as the rank increases. Similarly, considering the relationship with the training time, we selected 32 as the optimal rank for the model fine-tuning.

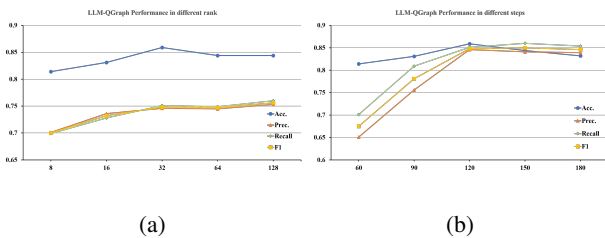


Fig. 4: Impact of various parameters on model performance

E. Case Study

In this section, we illustrate how the constructed threat intelligence query graphs can be applied to threat hunting through practical cases. By constructing threat intelligence query graphs based on attack analysis reports, the threat attack behaviors in text form can be transformed into graph form. Figure 5 shows the threat behavior query graph constructed according to the intrusion process of the TA4557 threat actor using the `more_eggs` backdoor. The TA4557 threat actor lure users to download the `more_eggs` malware through resumes and execute it using `ie4uinit.exe` and `msxsl.exe`, and then exploit vulnerabilities to move laterally to two servers, thereby spreading malicious payloads. Taking the initial access stage as an example, it can be clearly seen from the graph that the user accesses `johnshimkus[.]com` through the Firefox browser and download `johnshimkus.zip`. Subsequently, it is decompressed, enabling the `ie4uinit.exe` malware to be executed. The threat intelligence query graph contains the behavioral information of attackers. This behavioral information can be used as a query graph for capturing malicious behaviors in the system, allowing threat hunters to understand the behavioral characteristics of attackers and improve the accuracy of threat hunting.

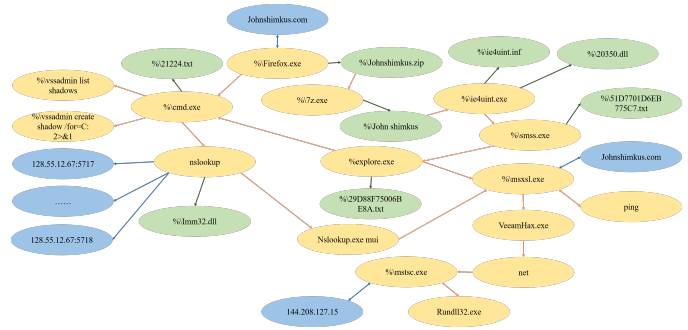


Fig. 5: A portion of the constructed query shows the intrusion process of the TA4557 threat actor using the more_eggs backdoor, where yellow nodes represent malicious processes, blue nodes represent networks, and green nodes represent files.

V. CONCLUSION AND DISCUSSION

In this paper, we propose an automated method for constructing threat intelligence query graphs based on large language models. By leveraging the few-shot learning ability of GPT and constructing corresponding prompts, we achieve the generation of simulated log datasets, thereby creating the datasets required for model fine-tuning and reducing the cost of manual annotation. Then, we fine-tune the Llama3 - 8B-Instruction model to enable it to analyze threat intelligence reports and generate log sequences that describe attack behaviors. Finally, we construct threat intelligence query graphs according to the proposed algorithm. Experiments have demonstrated that the query graphs constructed by our method are more effective in threat hunting applications compared to other methods. In future work, we will further explore rapid generation methods for log datasets corresponding to threat intelligence to provide more sufficient datasets for fine-tuning large language models.

REFERENCES

- [1] “M-Trends Report 2023,” <https://library.cyentia.com>.
- [2] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, “Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2019, p. 1795–1812.
- [3] R. Wei, L. Cai, L. Zhao, A. Yu, and D. Meng, “Deephunter: A graph neural network based approach for robust cyber threat hunting,” in *Security and Privacy in Communication Networks*. Springer, 2021, pp. 3–24.
- [4] K. Satvat, R. Gjomemo, and V. Venkatakrishnan, “Extractor: Extracting attack behavior from threat reports,” in *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021, pp. 598–615.
- [5] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang, “Attackg+:boosting attack knowledge graph construction with large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.04753>
- [6] Y. Hu, F. Zou, J. Han, X. Sun, and Y. Wang, “Llm-tikg: Threat intelligence knowledge graph construction utilizing large language model,” *Computers & Security*, vol. 145, p. 103999, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404824003043>
- [7] “MITRE ATT&CK®,” <https://attack.mitre.org/>.

- [8] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "Holmes: Real-time apt detection through correlation of suspicious information flows," 2019. [Online]. Available: <https://arxiv.org/abs/1810.01594>
- [9] H. Ding, J. Zhai, Y. Nan, and S. Ma, "AIRTAG: Towards automated attack investigation by unsupervised learning with log texts," in *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 373–390. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/ding-hailun-airtag>
- [10] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [12] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [13] E. Altinisik, F. Deniz, and H. T. Sencar, "Provg-searcher: A graph representation learning approach for efficient provenance graph search," 2023. [Online]. Available: <https://arxiv.org/abs/2309.03647>
- [14] P. Gao, X. Liu, E. Choi, B. Soman, C. Mishra, K. Farris, and D. Song, "A system for automated open-source threat intelligence gathering and management," 2021. [Online]. Available: <https://arxiv.org/abs/2101.07769>
- [15] D. Mu, A. Cuevas, L. Yang, H. Hu, X. Xing, B. Mao, and G. Wang, "Understanding the reproducibility of crowd-reported security vulnerabilities," in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, pp. 919–936. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/mu>
- [16] X. Liao, K. Yuan, X. Wang, Z. Li, L. Xing, and R. Beyah, "Acing the ioc game: Toward automatic discovery and analysis of open-source cyber threat intelligence," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 755–766. [Online]. Available: <https://doi.org/10.1145/2976749.2978315>
- [17] P. Gao, F. Shao, X. Liu, X. Xiao, Z. Qin, F. Xu, P. Mittal, S. R. Kulkarni, and D. Song, "Enabling efficient cyber threat hunting with cyber threat intelligence," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, 2021, pp. 193–204.
- [18] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, "Ttpdrill: Automatic and accurate extraction of threat actions from unstructured text of cti sources," in *Proceedings of the 33rd Annual Computer Security Applications Conference*, ser. ACSAC '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 103–115. [Online]. Available: <https://doi.org/10.1145/3134600.3134646>
- [19] V. Legoy, M. Caselli, C. Seifert, and A. Peter, "Automated retrieval of att&ck tactics and techniques for cyber threat reports," 2020. [Online]. Available: <https://arxiv.org/abs/2004.14322>
- [20] Z. Li, J. Zeng, Y. Chen, and Z. Liang, "Attackg: Constructing technique knowledge graph from cyber threat intelligence reports," 2022. [Online]. Available: <https://arxiv.org/abs/2111.07093>
- [21] M. Wüsch, A. Kucharavy, D. P. David, and A. Mermoud, "Llms perform poorly at concept extraction in cyber-security research literature," 2023. [Online]. Available: <https://arxiv.org/abs/2312.07110>
- [22] A. Zhong, D. Mo, G. Liu, J. Liu, Q. Lu, Q. Zhou, J. Wu, Q. Li, and Q. Wen, "Logparser-llm: Advancing efficient log parsing with large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2408.13727>